

文档分块

目的：文本分块即将大块切割成较小单元，

从文本分割本身来说，较小单元可以适配大模型的上下文长度限制；适配嵌入模型支持的文本块的长度限制；

从检索精度来说，小块信息更为集中，但过小可能不包含完整的信息，适用于短查询（如“北京的面积”）；过长则包含很多无关信息的噪声，嵌入向量无法精准表征核心语义信息，长查询（如“分析 AI 在医疗领域的应用现状与未来挑战”）需要中等粒度的块（段落级），因此应权衡，使得匹配准确率高，检索精度高；

从应用场景来说，不同类型的文档所适用的文本块的长度大小不一，如摘要需要段落级，问答需要句子级。

从处理效率来说，若分的太小，则数量上升，嵌入计算量增大，存储空间增大，检索匹配效率反而变慢，需要权衡；

文档清洗

从文档网页获取的初始数据有些是无益于训练甚至是有害的，此时便需要先对这些数据进行清洗，去除不想要的东西，优化数据以便后续的训练和处理等。一般来说，清洗的内容包括 HTML 标签，特殊字符和人为设置的停用词，此外还有大小写转换，词干提取和还原。这些操作一般可以通过特定的库和函数来实现，所以需要学习了解的就是库和函数的使用方法。

去除噪声：删除无关字符（如乱码、特殊符号）、广告文本、页眉页脚、重复内容等。

规范化格式：统一大小写、标点符号、日期/数字格式（如将 2023 年 10 月转为 2023-10）。

处理换行与空格：合并因 PDF/扫描文档换行导致的错误断句（例如用 regex 检测并修复）。

语言纠错：对 OCR 提取的文本进行拼写检查和语法修正（如使用 pySpellChecker 或预训练模型）。

去除 HTML 标签

当文档包含 HTML 格式时，需要去除其中的标签以获取纯文本内容。可以使用 **BeautifulSoup** 库来实现。

```
from bs4 import BeautifulSoup
```

```
html_doc = '<html>
<body>
<h1>这是一个标题</h1>
<p>这是一段文本。</p>
</body>
</html>'
soup = BeautifulSoup(html_doc, 'html.parser')
clean_text = soup.get_text()
print(clean_text)
优点
```

- 能够准确识别并去除各种 HTML 标签，保留文本信息。
- 操作简单，代码易于实现。

缺点

- 对于复杂的 HTML 结构，可能会出现一些误判或遗漏。
- 需要额外安装 BeautifulSoup 库。

去除特殊字符

使用正则表达式（详细解释一下？）可以去除文档中的特殊字符，如标点符号、数字、空格等。

```
import re
text = "Hello! 123 World@#"
#r'[]'中括号里填写要删除的字符即可，如果想要保留字符，则在前面加个^符号。
clean_text = re.sub(r'^[a-zA-Z\s]', "", text)
print(clean_text)
优点
```

- 灵活性高，可以根据需要自定义正则表达式来匹配不同的特殊字符。
- 可以快速批量处理文本。

缺点

- 正则表达式的编写需要一定的技巧，复杂的匹配规则可能会导致性能下降。
- 可能会误删一些有用的字符，需要谨慎调整正则表达式。

去除停用词

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('stopwords')
```

```
nltk.download('punkt')
```

```
text = "This is a sample sentence."  
stop_words = set(stopwords.words('english'))  
word_tokens = word_tokenize(text)  
filtered_text = [w for w in word_tokens if not w in stop_words]  
clean_text = ''.join(filtered_text)  
print(clean_text)
```

停用词是指在文本中频繁出现但对语义理解没有太大帮助的词汇，如“的”、“是”、“和”等。可以使用 **nltk 库中的停用词列表** 来去除这些词汇。

优点

- 可以减少文本的噪声，提高模型的处理效率。
- 有助于突出文本的关键信息。

缺点

- 停用词列表可能需要根据具体的应用场景进行调整，不同领域的停用词可能不同。
- 去除停用词可能会导致一些语义信息的丢失。

词干提取和词形还原

词干提取：将单词**还原为其词干形式**，不考虑单词的词性和语义。例如，“running”会被提取为“run”。

词形还原：根据单词的词性将其还原为字典中的形式，如“better”会被还原为“good”。

PorterStemmer 是一种词干提取器，它可以将单词还原为其词干形式。例如，“running”的词干是“run”，“flies”的词干是“fli”。

```
from nltk.stem import PorterStemmer  
#word_tokenize 函数用于将文本分割成单个的单词或标记（token）  
from nltk.tokenize import word_tokenize  
#调用 word_tokenize 函数，将 text 字符串分割成单个的单词或标记，并将结果  
存储在 word_tokens 列表中。在这个例子中，word_tokens 列表将包含 ['running',  
'quickly']。  
text = "running quickly"  
stemmer = PorterStemmer()  
word_tokens = word_tokenize(text)  
stemmed_text = [stemmer.stem(w) for w in word_tokens]  
clean_text = ''.join(stemmed_text)  
print(clean_text)#run quick
```

```

import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
nltk.download('wordnet')
nltk.download('omw-1.4')

text = "better"
lemmatizer = WordNetLemmatizer()
word_tokens = word_tokenize(text)
lemmatized_text = [lemmatizer.lemmatize(w) for w in word_tokens]
clean_text = ''.join(lemmatized_text)
print(clean_text) #good
优点

```

- 可以减少词汇的多样性，提高模型的泛化能力。
- 有助于发现文本中的语义关联。

缺点

- 词干提取可能会产生一些无意义的词干，影响语义理解。
- 词形还原需要更复杂的词性标注，处理速度相对较慢。

转换为小写

将文档中的所有字母转换为小写，统一文本的大小写，避免因大小写不同而导致的语义差异，但对于一些需要区分大小写的场景（如专有名词，apple 和 Apple），可能会丢失重要信息。

```

text = "Hello World"
clean_text = text.lower()
print(clean_text) #hello world

```

文档分割与智能分块策略

固定长度分割

按固定长度将文档分割成多个子文档，每个子文档的长度基本相同（最后一个子文档可能长度不足）。方法很简单，就是将变量以设定分割长度增加，然后将获取文本对应区间的内容。[text[i:i+length] for i in range(0, len(text), length)]

优点是：

- 简单直观，易于实现。

- 分割结果长度均匀，便于后续处理。

缺点是：

- 可能会在语义不完整的地方进行分割，影响语义连贯性。

`class FixedSizeChunker(BaseChunker):`

`"""固定长度分块器（支持重叠窗口）"""`

```
def __init__(self, chunk_size: int = 512, overlap: int = 100, **kwargs):
    super().__init__(chunk_size, **kwargs)
    self.overlap = overlap
```

```
def chunk(self, text: str) -> List[str]:
    tokens = self.tokenizer.encode(text)
    chunks = []
    start = 0
    while start < len(tokens):
        end = min(start + self.chunk_size, len(tokens))
        chunk_tokens = tokens[start:end]
        chunks.append(self.tokenizer.decode(chunk_tokens))
        start += self.chunk_size - self.overlap
    return chunks
```

递归字符分块（Recursive Character Chunking）

按照预设的分隔符优先级（段落，换行，句号，逗号，叹号等）递归切分文本，直到每个块达到预设的 Token / 字符阈值。核心逻辑是优先使用天然语义分隔符切分，最大程度避免破坏语义结构，是 LangChain 等框架的默认方案，也是工业界最常用的基线方案。中文场景常用分隔符优先级（从高到低）：["\n\n", "\n", "。", "!", " ", "? ", " ", ""]

以带段落结构的扫地机器人说明为例：

XX 智能扫地机器人具备 LDS 激光导航功能，可实现 10 米范围内的精准建图，支持全屋分区清扫、禁区设置，单次续航可达 180 分钟，适配 150 m²以内的户型。

同时搭载 2700Pa 大吸力，可清理浮尘、毛发、颗粒物，支持自动集尘，30 天无需手动倒垃圾。

常见故障排查：

1. 无法建图：请检查激光雷达是否有遮挡，重启设备后重试；
2. 吸力不足：请清理滚刷与尘盒，检查吸力档位是否设置正确。

设置规则：块上限 200 字符，按上述分隔符优先级递归切分，结果：

- 块 1: XX 智能扫地机器人具备 LDS 激光导航功能, 可实现 10 米范围内的精准建图, 支持全屋分区清扫、禁区设置, 单次续航可达 180 分钟, 适配 150 m²以内的户型。
- 块 2: 同时搭载 2700Pa 大吸力, 可清理浮尘、毛发、颗粒物, 支持自动集尘, 30 天无需手动倒垃圾。
- 块 3: 常见故障排查: 1. 无法建图: 请检查激光雷达是否有遮挡, 重启设备后重试; 2. 吸力不足: 请清理滚刷与尘盒, 检查吸力档位是否设置正确。

(固定长度分块大概率会将故障排查的条目从中间切断, 递归分块则优先保留段落和列表的完整性)

优点

最大程度保留语义完整性, 优先按段落、句子等天然语义边界切分, 效果远优于固定长度分块;

灵活性高, 可根据文本类型自定义分隔符, 比如代码场景可新增 `def`、`class`、`{}` 等分隔符;

缺点

对无标点、无段落结构的文本 (OCR 识别文本、口语转写文本) 适配性差, 效果会退化到固定长度分块;

无法识别跨段落的语义关联, 相邻段落若为同一主题, 仍会被强制切分, 导致语义碎片化;

绝大多数通用 RAG 场景, 包括结构化 Word/PDF 文档、博客、论文、产品手册等, 是工业界首选的通用基线方案。

绝大多数通用 RAG 场景, 包括结构化 Word/PDF 文档、博客、论文、产品手册等, 是工业界首选的通用基线方案。

`class RecursiveCharacterChunker(BaseChunker):`

`"""递归字符分块器 (按语义分隔符优先级切分) """`

```
def __init__(
    self,
    chunk_size: int = 512,
    separators: Optional[List[str]] = None,
    **kwargs
):
    super().__init__(chunk_size, **kwargs)
    self.separators = separators or ["\n\n", "\n", "。", "!", "?", ",", " ", ""]
```

```
def _split_text(self, text: str, separator: str) -> List[str]:
    if not separator:
        return list(text)
    splits = text.split(separator)
    return [s + separator for s in splits[:-1]] + ([splits[-1]] if splits[-1] else [])
```

```

def _merge_splits(self, splits: List[str]) -> List[str]:
    chunks = []
    current_chunk = ""
    for split in splits:
        if self.count_tokens(current_chunk + split) <= self.chunk_size:
            current_chunk += split
        else:
            if current_chunk:
                chunks.append(current_chunk)
            if self.count_tokens(split) > self.chunk_size:
                chunks.extend(self._merge_splits(self._split_text(split,
self.separators[-1])))
            else:
                current_chunk = split
    if current_chunk:
        chunks.append(current_chunk)
    return chunks

def chunk(self, text: str) -> List[str]:
    final_chunks = []
    current_splits = [text]
    for sep in self.separators:
        new_splits = []
        for split in current_splits:
            if self.count_tokens(split) > self.chunk_size:
                new_splits.extend(self._split_text(split, sep))
            else:
                new_splits.append(split)
        current_splits = new_splits
    return self._merge_splits(current_splits)

```

基于语义的分块（Semantic/Embedding-based Chunking）

先将文本拆分为句子 / 短句，生成每个短句的嵌入向量（Embedding），计算相邻句子的向量相似度；当相似度低于预设阈值时，在此处切分，将语义高度相似的连续句子合并为一个块。切分的唯一依据是语义主题的一致性，完全贴合语义边界。

以大模型技术文本为例：

Transformer 架构是大语言模型的核心基础。它由编码器和解码器两部分组成，核心创新是自注意力机制。自注意力机制可以让模型在处理每个词时，关注到文本中所有相关的词，解决了 RNN 的长距离依赖问题。BERT 是基于 Transformer 编码器的预训练模型，擅长理解类任务。GPT 系列是基于 Transformer 解码器的自回归模型，擅长生成类任务。

步骤 1：拆分为独立句子

1. Transformer 架构是大语言模型的核心基础。
2. 它由编码器和解码器两部分组成，核心创新是自注意力机制。
3. 自注意力机制可以让模型在处理每个词时，关注到文本中所有相关的词，解决了 RNN 的长距离依赖问题。
4. BERT 是基于 Transformer 编码器的预训练模型，擅长理解类任务。
5. GPT 系列是基于 Transformer 解码器的自回归模型，擅长生成类任务。

步骤 2：生成句子嵌入，计算相邻相似度句 1 - 句 2 相似度 0.92，句 2 - 句 3 相似度 0.88，句 3 - 句 4 相似度 0.65，句 4 - 句 5 相似度 0.85。

步骤 3：设置相似度阈值 0.7，低于阈值处切分，最终结果：

- 块 1：Transformer 架构是大语言模型的核心基础。它由编码器和解码器两部分组成，核心创新是自注意力机制。自注意力机制可以让模型在处理每个词时，关注到文本中所有相关的词，解决了 RNN 的长距离依赖问题。
- 块 2：BERT 是基于 Transformer 编码器的预训练模型，擅长理解类任务。GPT 系列是基于 Transformer 解码器的自回归模型，擅长生成类任务。

优点

切分完全贴合语义边界，块内主题高度统一，召回精度大幅提升；
可识别跨段落的语义关联，同一主题的相邻段落会自动合并，避免语义碎片化。

缺点

成本高，需要提前对所有句子生成嵌入向量，增加了算力开销和处理时长；
阈值调优难度大，阈值过高导致块过大、细节丢失，阈值过低导致块过碎、上下文不足；

可解释性差，切分位置由向量相似度决定，出现问题时难以排查和调试。

优化方法：

普通语义分块完全依赖相邻句子间的相似度（局部判断），而不考虑非相邻句子的全局语义关联。如果出现语义跳跃的情况，无法很好地分块，优化方式一：则计算每个句子和当前块中所有句子的相似度，但计算量会大大提高；二：则先使用低阈值进行聚类，把所有相似的句子先聚到一块，再进行高精度聚类。

2.语义分块和后面的文本嵌入应合二为一，减少嵌入向量的消耗。

对召回精度要求极高的场景，比如法律文书、医疗病历、科研论文、专业知识库，以及无格式文本的处理。

class SemanticChunker(BaseChunker):

"""语义分块器（基于嵌入向量相似度）"""

```
def __init__(
    self,
    chunk_size: int = 512,
    embedding_model: str = "all-MiniLM-L6-v2",
```



```

        similarity_threshold: float = 0.7,
        **kwargs
    ):
        super().__init__(chunk_size, **kwargs)
        self.embedding_model = SentenceTransformer(embedding_model)
        self.similarity_threshold = similarity_threshold

    def _cosine_similarity(self, vec1: np.ndarray, vec2: np.ndarray) -> float:
        return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

    def chunk(self, text: str) -> List[str]:
        sentences = sent_tokenize(text)
        if not sentences:
            return []
        embeddings = self.embedding_model.encode(sentences)
        chunks = []
        current_chunk = [sentences[0]]
        current_embedding = embeddings[0]
        for i in range(1, len(sentences)):
            sim = self._cosine_similarity(current_embedding, embeddings[i])
            if sim >= self.similarity_threshold and self.count_tokens("
.join(current_chunk + [sentences[i]])) <= self.chunk_size:
                current_chunk.append(sentences[i])
                current_embedding = np.mean(embeddings[:i+1], axis=0)
            else:
                chunks.append(" ".join(current_chunk))
                current_chunk = [sentences[i]]
                current_embedding = embeddings[i]
        if current_chunk:
            chunks.append(" ".join(current_chunk))
        return chunks

```

基于文档结构 / 格式的分块 (Structure-aware Chunking)

先解析文档的原生结构与格式元素（标题层级、章节、段落、列表、表格、代码块、图片注释等），按照文档的逻辑层级进行切分，优先保留章节、小节的完整性，同时区分不同类型的内容元素，是企业级知识库的核心方案。

以 Markdown 格式的平台用户手册为例：

```

# XX 平台用户手册## 一、账号注册### 1.1 个人账号注册
个人账号注册需填写手机号、验证码，设置 6-16 位密码，完成实名认证后即可使用。### 1.2 企业账号注册

```

企业账号需上传营业执照、法人身份证，填写企业对公账户信息，审核周期 1-3 个工作日。## 二、功能使用### 2.1 数据上传

支持 Excel、CSV 格式文件上传，单文件最大支持 1GB，单次最多上传 10 个文件。### 2.2 数据可视化

支持折线图、柱状图、饼图等多种图表类型，可自定义坐标轴、配色、图例。
设置规则：按二级标题为核心切分单元，三级标题为辅助粒度，保留标题层级，切分结果：

- 块 1：# XX 平台用户手册 ## 一、账号注册 ### 1.1 个人账号注册 个人账号注册需填写手机号、验证码，设置 6-16 位密码，完成实名认证后即可使用。
- 块 2：# XX 平台用户手册 ## 一、账号注册 ### 1.2 企业账号注册 企业账号需上传营业执照、法人身份证，填写企业对公账户信息，审核周期 1-3 个工作日。
- 块 3：# XX 平台用户手册 ## 二、功能使用 ### 2.1 数据上传 支持 Excel、CSV 格式文件上传，单文件最大支持 1GB，单次最多上传 10 个文件。
- 块 4：# XX 平台用户手册 ## 二、功能使用 ### 2.2 数据可视化 支持折线图、柱状图、饼图等多种图表类型，可自定义坐标轴、配色、图例。

同时，文档中的表格、代码块会被作为独立块处理，避免与正文混切。

优点

完美保留文档原生逻辑结构和层级关系，块内内容属于同一章节 / 主题，语义完整性拉满；

检索时可利用标题层级加权，比如用户搜索“企业账号注册”，可精准召回对应小节，同时携带父级标题，帮助大模型更好理解上下文；

缺点

对文档解析能力要求极高，扫描版 PDF、无样式的混乱文档、乱码 HTML 的解析效果极差；

实现复杂度高，需要对接专业文档解析工具（Unstructured、PyMuPDF 等），不同格式文档需单独适配；

有明确结构的企业文档、产品手册、技术文档、政策文件、教材、电子书、Markdown/HTML 网页，是企业级知识库的首选方案。

```
class MarkdownStructureChunker(BaseChunker):
```

```
    """Markdown 结构感知分块器（按标题层级切分）"""
```

```

def __init__(self, chunk_size: int = 512, **kwargs):
    super().__init__(chunk_size, **kwargs)
    self._level_pattern = re.compile(r"^(#{1,6})s")

def _parse_markdown(self, text: str) -> List[dict]:
    lines = text.split("\n")
    elements = []
    current_heading = ""
    current_content = []
    for line in lines:
        match = self._level_pattern.match(line)
        if match:
            if current_heading or current_content:
                elements.append({"heading": current_heading, "content":
"\n".join(current_content).strip()})
                current_heading = line.strip()
                current_content = []
            else:
                current_content.append(line)
        if current_heading or current_content:
            elements.append({"heading": current_heading, "content":
"\n".join(current_content).strip()})
    return elements

def chunk(self, text: str) -> List[str]:
    elements = self._parse_markdown(text)
    chunks = []
    current_chunk = ""
    for elem in elements:
        block = f"{elem['heading']}\n{elem['content']}".strip()
        if self.count_tokens(current_chunk + "\n" + block) <= self.chunk_size:
            current_chunk = f"{current_chunk}\n{block}".strip()
        else:
            if current_chunk:
                chunks.append(current_chunk)
            if self.count_tokens(block) > self.chunk_size:
                sub_chunker =
RecursiveCharacterChunker(chunk_size=self.chunk_size)
                chunks.extend(sub_chunker.chunk(block))
            else:
                current_chunk = block
    if current_chunk:
        chunks.append(current_chunk)
    return chunks

```

基于规则 / 知识单元的分块 (Rule-based / Knowledge Unit Chunking)

针对特定领域的标准化文档，自定义专属切分规则，按照知识的最小完整单元进行切分，比如法律的法条、金融的财报条目、客服的 QA 对、医疗的病历模块，是垂直领域知识库的核心方案。

以《民法典》合同编标准化条文为例：

第五百零九条 当事人应当按照约定全面履行自己的义务。当事人应当遵循诚信原则，根据合同的性质、目的和交易习惯履行通知、协助、保密等义务。当事人在履行合同过程中，应当避免浪费资源、污染环境和破坏生态。

第五百一十条 合同生效后，当事人就质量、价款或者报酬、履行地点等内容没有约定或者约定不明确的，可以协议补充；不能达成补充协议的，按照合同相关条款或者交易习惯确定。

自定义规则：以“第 X 条”为核心分隔符，单条法条为一个完整知识单元，切分结果：

- 块 1：第五百零九条 当事人应当按照约定全面履行自己的义务。当事人应当遵循诚信原则，根据合同的性质、目的和交易习惯履行通知、协助、保密等义务。当事人在履行合同过程中，应当避免浪费资源、污染环境和破坏生态。
- 块 2：第五百一十条 合同生效后，当事人就质量、价款或者报酬、履行地点等内容没有约定或者约定不明确的，可以协议补充；不能达成补充协议的，按照合同相关条款或者交易习惯确定。

优点

完全贴合领域知识特点，切分后的块是完整的知识单元，无语义断裂，召回与生成效果拉满；

针对性极强，针对垂直领域文档深度优化，适配性远高于通用分块方法；

缺点

通用性极差，一套规则仅适配一类文档，更换领域 / 格式需重新开发；

开发成本高，需要开发人员同时熟悉领域知识和文档格式，定制化开发成本高；

```
class RuleBasedChunker(BaseChunker):
```

```
    """规则分块器（基于自定义正则表达式）"""
```

```
    def __init__(self, chunk_size: int = 512, pattern: str = r"第\d+条", **kwargs):
        super().__init__(chunk_size, **kwargs)
        self.pattern = re.compile(pattern)
```

```
    def chunk(self, text: str) -> List[str]:
        splits = self.pattern.split(text)
        matches = self.pattern.findall(text)
```

```

chunks = []
current_chunk = ""
for i in range(len(splits)):
    block = (matches[i-1] if i > 0 else "") + splits[i]
    if self.count_tokens(current_chunk + block) <= self.chunk_size:
        current_chunk += block
    else:
        if current_chunk:
            chunks.append(current_chunk)
        if self.count_tokens(block) > self.chunk_size:
            sub_chunker =
RecursiveCharacterChunker(chunk_size=self.chunk_size)
            chunks.extend(sub_chunker.chunk(block))
        else:
            current_chunk = block
if current_chunk:
    chunks.append(current_chunk)
return chunks

```

代码演示

```

import abc
import re
from typing import List, Optional
import tiktoken
import nltk
from nltk.tokenize
import sent_tokenize
from sentence_transformers import SentenceTransformer
import numpy as np
from mistune import markdown as parse_markdown

class BaseChunker(abc.ABC):
    """分块器基类：定义统一接口"""

    def __init__(self, chunk_size: int = 512, tokenizer_model: str = "gpt-3.5-turbo"):
        self.chunk_size = chunk_size
        self.tokenizer = tiktoken.encoding_for_model(tokenizer_model)

    def count_tokens(self, text: str) -> int:

```

```
"""计算文本的 Token 数"""
return len(self.tokenizer.encode(text))
```

```
@abc.abstractmethod
def chunk(self, text: str) -> List[str]:
    """核心分块方法：输入原始文本，输出文本块列表"""
    pass
```

```
sample_text = """
# XX 智能扫地机器人手册
## 一、核心功能
XX 智能扫地机器人具备 LDS 激光导航功能，可实现 10 米范围内的精准建图，
支持全屋分区清扫、禁区设置，单次续航可达 180 分钟，适配 150 m² 以内的户型。
同时搭载 2700Pa 大吸力，可清理浮尘、毛发、颗粒物，支持自动集尘，30 天无
需手动倒垃圾。
```

```
## 二、故障排查
### 2.1 无法建图
请检查激光雷达是否有遮挡，重启设备后重试。
### 2.2 吸力不足
请清理滚刷与尘盒，检查吸力档位是否设置正确。
"""
```

```
# 1. 固定长度分块 print("=== 固定长度分块 ===")
fixed_chunker = FixedSizeChunker(chunk_size=100, overlap=20)for i, chunk in
enumerate(fixed_chunker.chunk(sample_text)):
    print(f'块 {i+1}: \n{chunk} \n')
# 2. 递归字符分块 print("=== 递归字符分块 ===")
recursive_chunker = RecursiveCharacterChunker(chunk_size=200)for i, chunk in
enumerate(recursive_chunker.chunk(sample_text)):
    print(f'块 {i+1}: \n{chunk} \n')
# 3. 语义分块 print("=== 语义分块 ===")
semantic_chunker = SemanticChunker(chunk_size=200, similarity_threshold=0.7)for
i, chunk in enumerate(semantic_chunker.chunk(sample_text)):
    print(f'块 {i+1}: \n{chunk} \n')
# 4. Markdown 结构分块 print("=== Markdown 结构分块 ===")
md_chunker = MarkdownStructureChunker(chunk_size=300)for i, chunk in
enumerate(md_chunker.chunk(sample_text)):
    print(f'块 {i+1}: \n{chunk} \n')
```

```
# 5. 规则分块（以法律条文为例）
```

```
legal_text = """
第五百零九条 当事人应当按照约定全面履行自己的义务。当事人应当遵循诚信
原则，根据合同的性质、目的和交易习惯履行通知、协助、保密等义务。
```

第五百一十条 合同生效后，当事人就质量、价款或者报酬、履行地点等内容没有约定或者约定不明确的，可以协议补充；不能达成补充协议的，按照合同相关条款或者交易习惯确定。

```
"""print("=== 规则分块（法律条文） ===")
rule_chunker = RuleBasedChunker(chunk_size=200, pattern=r"第\d+条")
for i, chunk in enumerate(rule_chunker.chunk(legal_text)):
    print(f'块{i+1}:\n{chunk}\n')
```

文档结构化

结构化的目的是为了给文档内容更多维度的信息，结构化的步骤是给文档添加元数据，比如页码，段落，语义标签，作者，文档等信息。所以需要考虑的就是 2 部分，一是这些信息从哪里来，如何获取；第二部分就是这些信息如何添加传递到提取分割后的文档块里。

首先来看第一部分，信息从哪来，如何获取，先看一下这些元数据一般有哪些，来自哪里，

元数据主要来自：

类型	来源示例	典型字段
文档级	- 文件属性（名称、路径、创建时间） - 文档格式信息（PDF/Word 等）	source, file_type, created_date
内容结构	- 文档结构解析（章节标题、页码） - 内容分析结果（主题分类）	page, section, heading_level, topic
处理过程	- 分块操作记录（分块时间） - 处理工具版本	chunk_time, processor_version
外部关联	- 数据库关联信息 - 用户标注数据	doc_id, user_tags, access_level

元数据与文本块的对应实现

文档分割阶段携带

```
def pdf_chunking(file_path):
    chunks = []
    for page_num, page_layout in enumerate(extract_pages(file_path)):
        # 提取原始元数据
        metadata = {
            "source": file_path,          # str
            "page": page_num + 1,         # int
            "total_pages": len(extract_pages(file_path)) # int
        }
```

```
# 文本提取与分块
text = extract_text_from_page(page_layout) # str
chunks.append((text, metadata)) # Tuple[str, Dict]
return chunks # List[Tuple[str, Dict]]
```

不如语冰